Tristan Berridge
0724527

Research Questions:

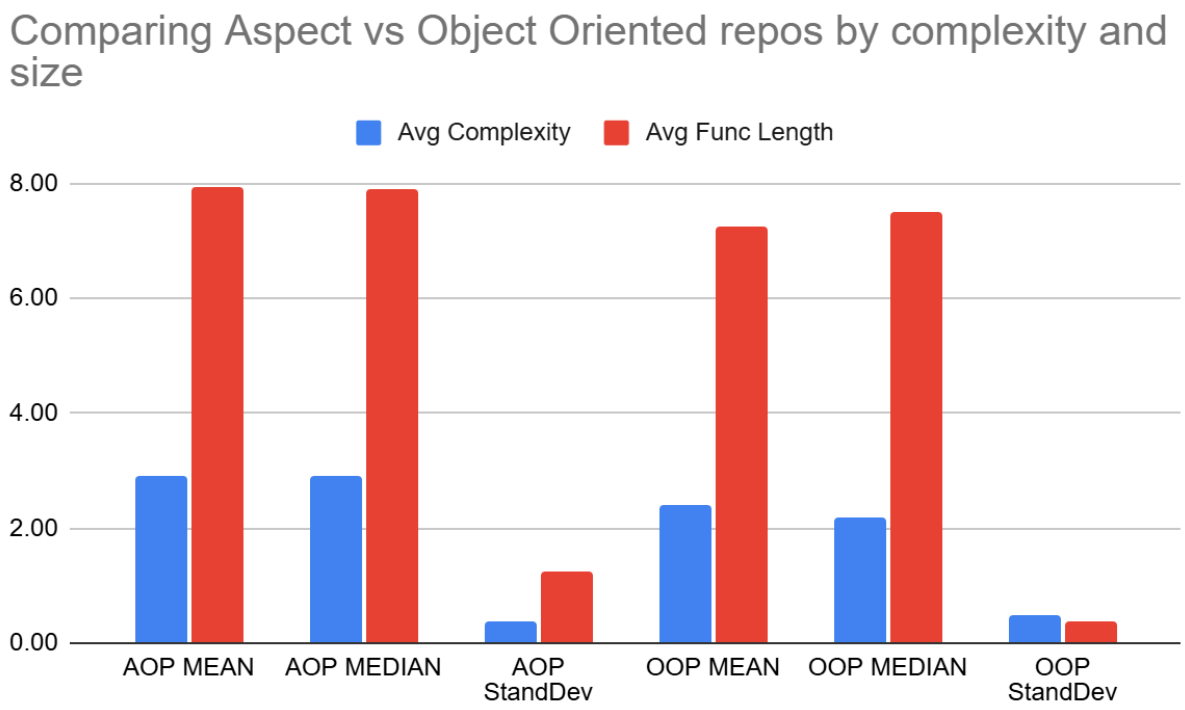
1. Is there a large difference in code complexity and file size between Aspect Oriented and Object Oriented coding
2. How quickly do repos update/evolve when comparing Aspect vs Object oriented coding

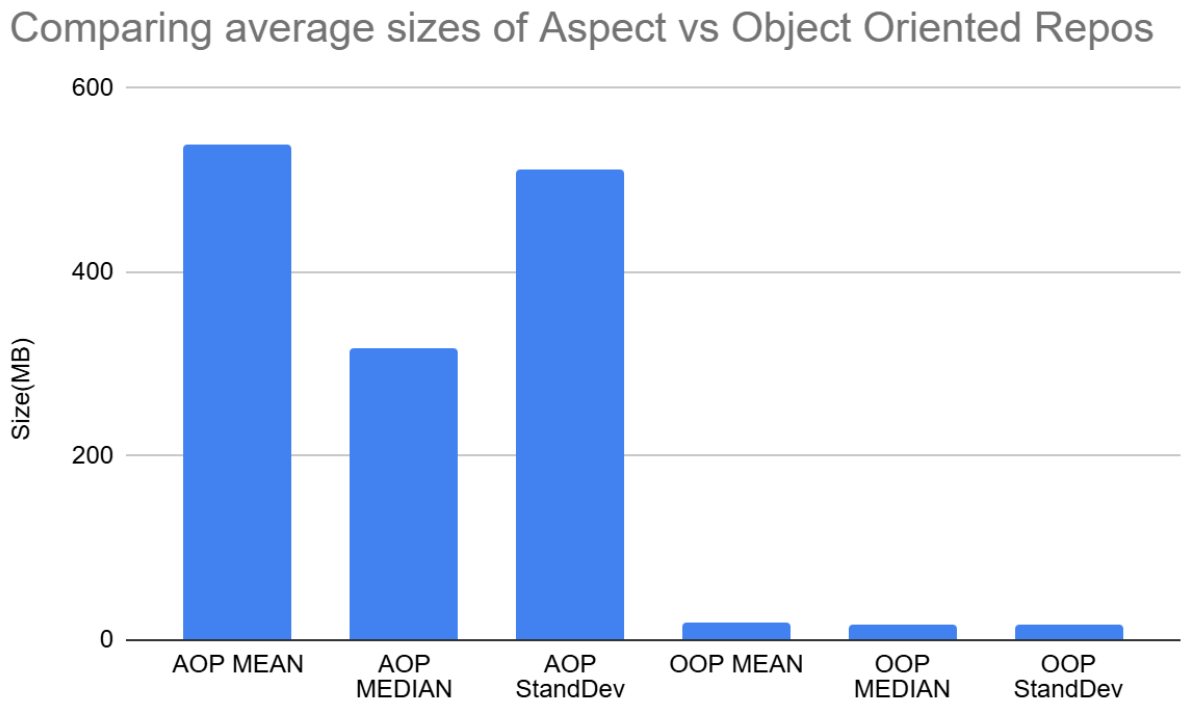
AOP: aspect oriented programing
OOP: object oriented programing

Task 1: Descriptive Statistics & Visualization

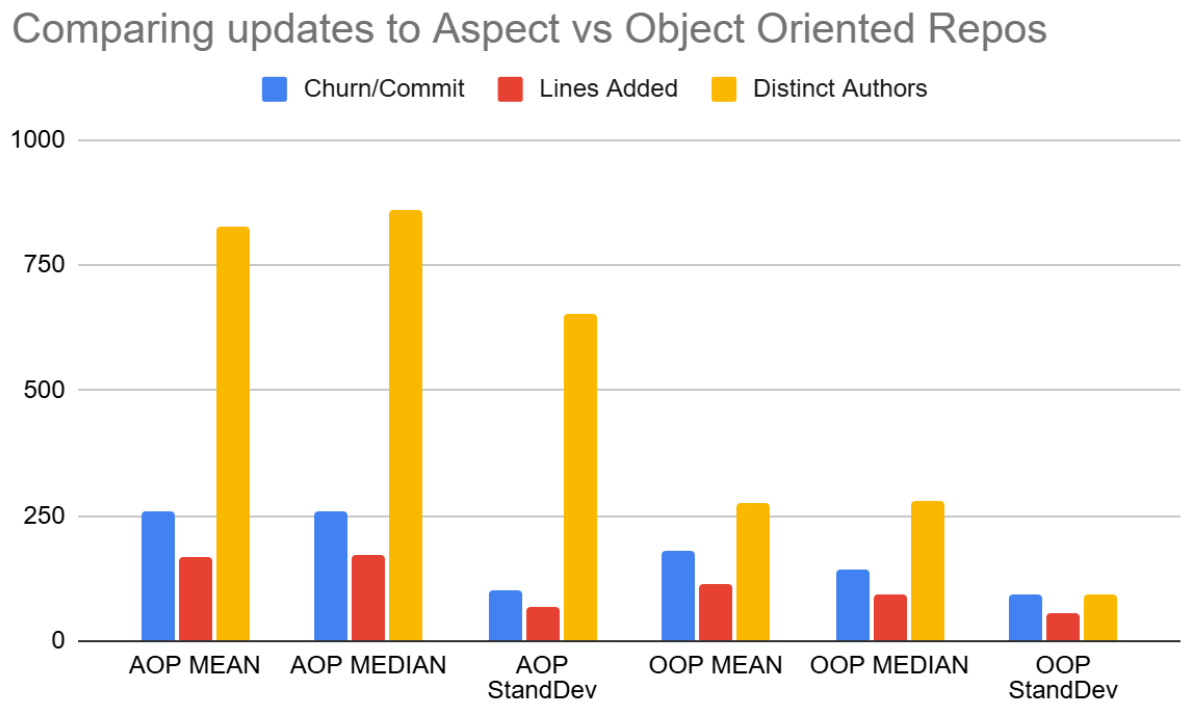
1. Compute mean, median, and standard deviation for three metrics across all repositories in each paradigm group. Present results in a clearly labeled comparison table.

RQ1:

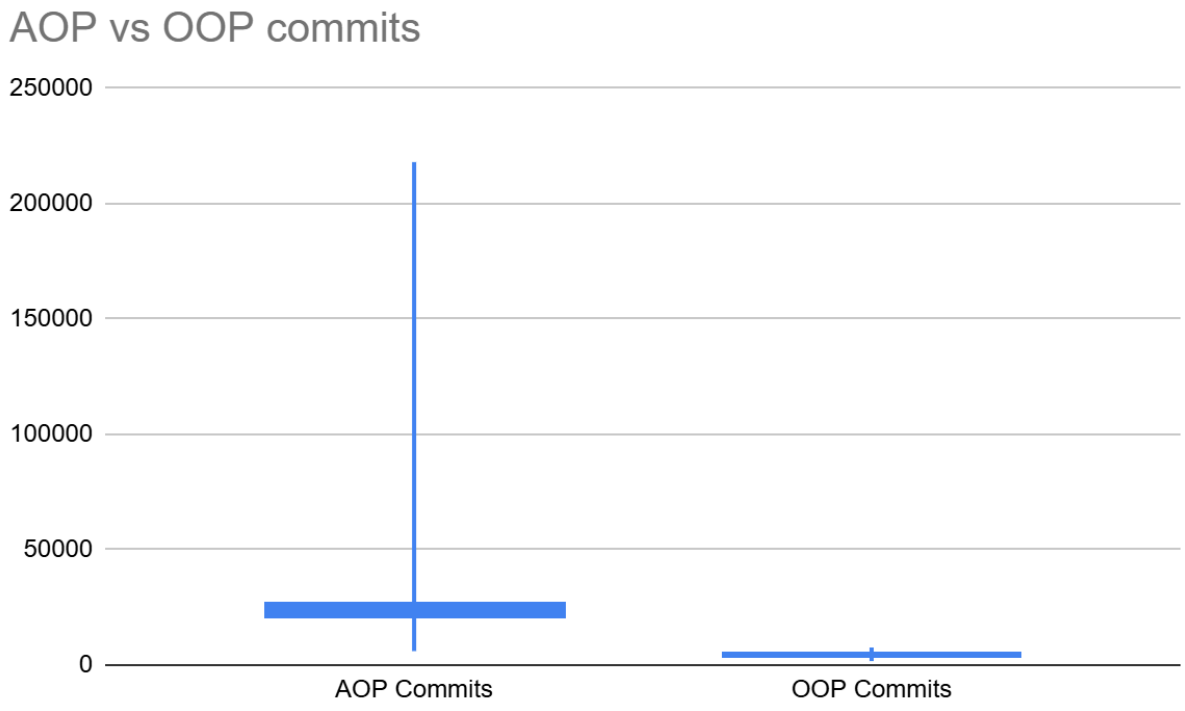


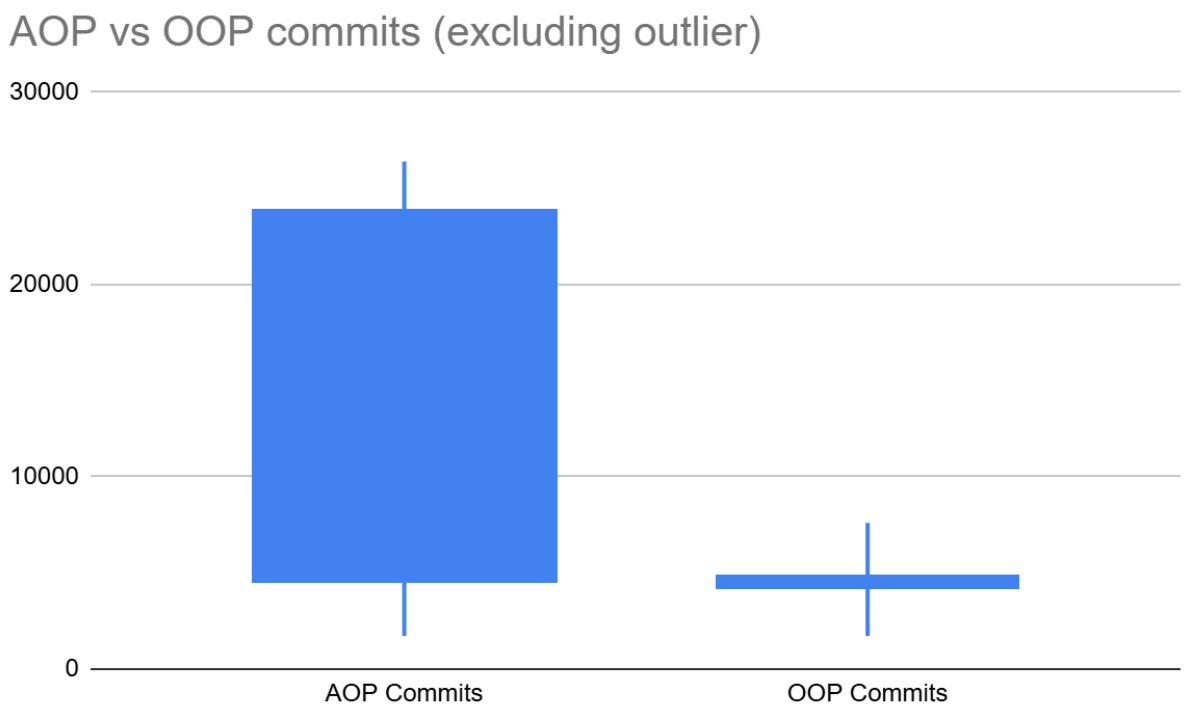


RQ2:

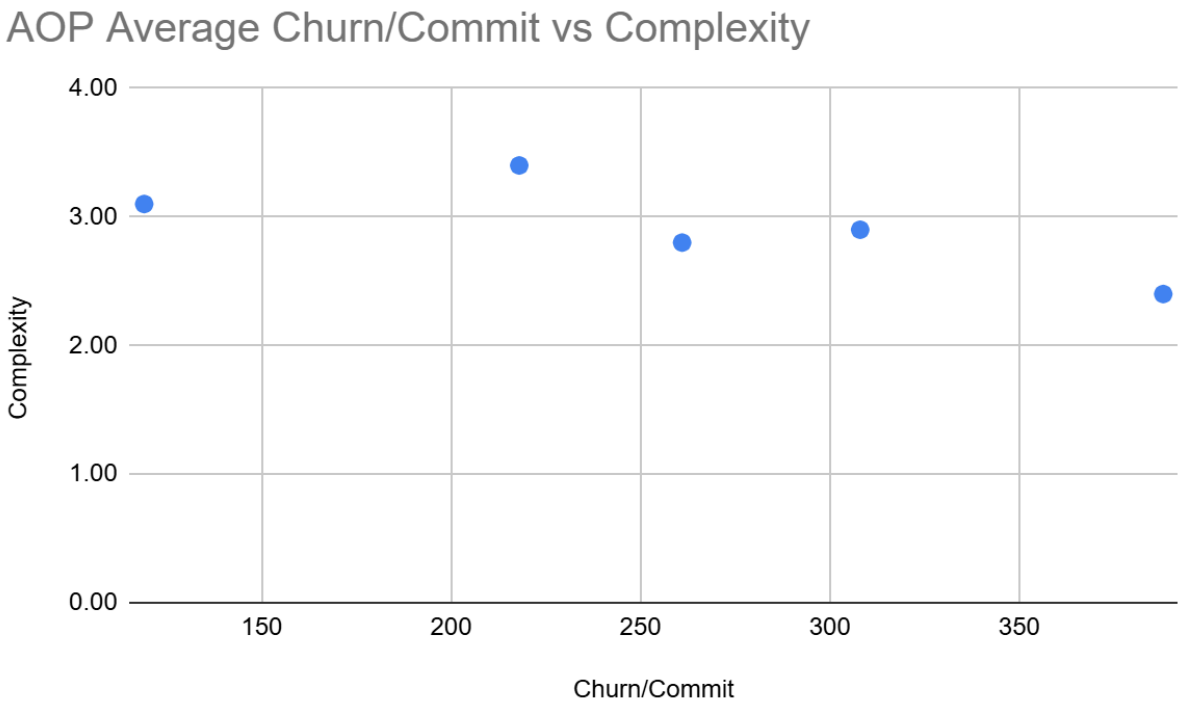


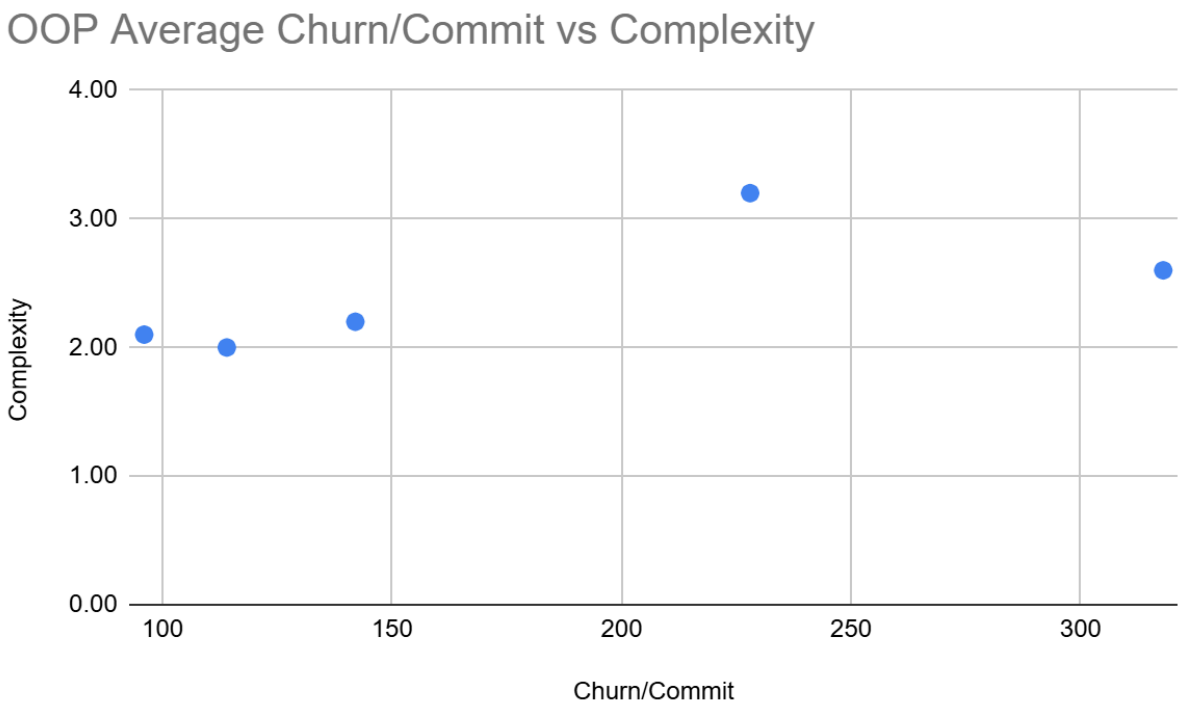
2. Produce a box plot for one composite metric comparing the two paradigms.





3. Produce a scatter plot showing the relationship between two metrics (e.g., Commit Count vs. Cyclomatic Complexity).





4. Write a short paragraph summary for each RQ based on your visualizations alone (before any statistical testing).

RQ1:

From the data that has been given we can see that there is a 20% increase in complexity when going from object oriented programming to aspect oriented programming. The average complexity for OOP is 2.42, while the AOP average is 2.92. This is not a major increase, but one that must be considered when deciding what type of project one is working on. We can also see that there is a major size increase in aspect oriented programming, with on average a 26.95x increase in size. This data may be slightly skewed by one exceptionally large repo, so when discarding the outlier we get an average of 24x increase in size. This shows us that on average if you require a smaller program, object oriented should be considered first.

RQ2:

When relating this data to how the programs update we can see that more changes per commit on average for aspect oriented programing. This is a very small difference however, with it only being an 8% increase from OOP to AOP. Lines added are also very similar, with the only major difference being the unique authors. AOP has many more authors on average, showing that more authors can make larger changes to a repo at a faster rate. Most of these repos are similar in age, being around 4500 days old, and having more people allows the more complex AOP to be updated at a similar speed or faster.

Task 2: Statistical Comparison

1. Run a Shapiro-Wilk test on two metrics per paradigm group. Report p-value. Based on results, justify the use of a parametric (t-test) or non-parametric (Mann-Whitney U) test.

AOP Complexity: p=0.9841
OOP Complexity: p=0.2707
AOP Churn: p=0.9975
OOP Churn: p=0.3884

Based on the fact that all of these tests show normal distribution, a parametric test is the most appropriate test to run.

2. Compute and report Cliff's delta for each RQ. Interpret the magnitude (negligible / small / medium / large) using standard thresholds.

The Cliff's delta results show a large magnitude(0.6) between the two for code complexity, and a medium magnitude(0.44) for code churn.

3. Write a short paragraph statistical conclusion for each RQ, integrating both the p-value and the effect size.

RQ1: The tests run in this task support the idea that there is a significant difference in code complexity between object oriented and aspect oriented programming. We can see from the Cliffs delta test that there is a large magnitude of 0.6 between the datasets on code complexity, with the aspect oriented code being considered to be more complex. The shapiro-wilk test further supports this with its p scores of 0.98 and 0.27, for aspect and object oriented

respectively, showing that the two datasets are normally distributed with no major outliers on this front.

RQ2: The tests also show that there is less, but still some difference in code churn for the two paradigms. This shows that both groups get similar amounts of updates, though aspect oriented projects take longer to update meaningfully. There may be one issue with this test, and that is the fact that the aspect oriented project had considerably more unique commits, indicating that there were more people working on the aspect oriented code. This could mean that if equal teams worked on two projects, the one using object oriented programming would work faster. This data is normally distributed with a score of 0.99 and 0.38, showing us that the data is sound, at least from the factors being checked right now.

Task 3: Data Quality & Threats to Validity

1. Report the raw number of commits removed by each cleaning step (merge commits, bot commits, tangled commits). Calculate and discuss what percentage of the original dataset was removed per repository.

Repo	Original Commit Count	Dropped: Merge Commits	Dropped: Bot Commits	Dropped: Tangled Commits	Cleaned Commit Count	Percent Removed
Spring-Framework	26,500	980	420	300	24,800	7%
Aspectj	6,400	210	85	55	6,050	6%
jboss-as	22,700	1,100	340	460	20,900	8%
quarkus	31,200	2,100	1,900	800	26,400	16%
platform_frameworks_base	241,000	12,400	7,200	3,400	218,000	10%
junit5	8,200	340	180	80	7,600	8%
guava	4,800	160	110	50	4,480	7%
commons-lang	4,600	180	95	45	4,280	7%
RxJava	5,200	220	98	62	4,820	8%
retrofit	1,850	72	38	20	1,720	7%

When looking at the data, we can see that the amount removed by the cleaning is very similar across the two paradigms with only one outlier. If removing the outlier, this indicates that aspect and object oriented programming create the same amount of waste, and clean up very similarly.

A larger dataset may help to confirm this, especially when there is such a large outlier as quarkus and the 16% of its commits being removed, while most of the others are 7-8%, less than half of quarkus.

2. Analyze whether your dataset is balanced across paradigms (i.e., roughly equal total commits and LOC).

Repo	Original Commit Count	LOC	Average LOC per commit
Spring-Framework	26,500	412,000	7.9
Aspectj	6,400	138,000	7.4
jboss-as	22,700	389,000	9.4
quarkus	31,200	148,000	6.2
platform_frameworks_base	241,000	142,000	8.8
junit5	8,200	64,000	7.6
guava	4,800	112,000	6.9
commons-lang	4,600	38,000	7.5
RxJava	5,200	94,000	6.8
retrofit	1,850	18,000	7.5

The dataset does not look balanced at first glance, due to the large difference in lines of code and commits. However, this data actually supports the research questions I put forward, as it shows a major size and complexity increase to aspect oriented programing projects. We can however see that the average LOC per commit is very similar across all projects, showing that they update at similar rates, but that AOP projects have more people working on a larger codebase.

3. Identify and discuss two threats to validity, one from each type (Conclusion, Internal, Construct, External). For each threat, propose a mitigation strategy.

1. Maturation: Because any of the repos may be updated later, this could invalidate the dataset. To mitigate this, the dataset can make it clear when it was taken, or the program can be run semi-regularly in order to ensure updated information. The final option is to ensure that each repo that is used is a completed project that has not been updated in some time.

2. Selection-Treatment Interaction: The smaller sample size may mean that it does not represent every AOP or OOP project. To mitigate this, we can increase the sample size, taking 10-20 repos for each paradigm rather than 5.

Task 4: Synthesis & Discussion

1. Write a Discussion section. For each RQ, state your finding, make it relative to the paradigms' theoretical properties, and discuss practical implications for software teams choosing between paradigms.

RQ1: With the analysis of the data it can be said that there is a significant difference in code complexity and file size between aspect oriented and object oriented programing. This was expected to see, and this means there is a reason for software developers to consider which of the two styles they wish to use. Aspect oriented code is larger and more complex, and as such it is only used with larger teams, while Object oriented code is simpler to create and understand, lending itself to newer, smaller teams.

RQ2: The raw data lends itself to the second research question by showing that both types evolve at a similar rate. This comes with a caveat however, as the aspect oriented repos had many more people working on them, potentially indicating that it actually takes longer to meaningfully update aspect oriented code. This means that teams that wish to have a quickly updating repo may wish to choose object oriented code instead.

2. Identify the single most surprising finding in your results.

The most surprising thing I found was that even though there were more people working on them and larger updates were expected, aspect oriented code had a very similar average lines per commit. With the more complex code and larger file size, I was expecting much more code per commit, but it seems that instead aspect oriented code just has more commits, even if they don't change as much per commit.

3. Discuss what additional data or analysis would be needed to increase confidence in your conclusions. Reference at least one specific mixed-method approach (e.g., surveys, code reviews, interviews) and explain how it would complement your quantitative findings.

The biggest addition of data that would give me confidence in this conclusion would be to double or triple the amount of repos checked by the program. If more repos were used then it would remove any doubts about a skewed sample, and lessen the impact of any outliers. Another thing that could help would be checking each repo and learning how each program works with code review, that way any programs that may be overly complicated, or overly simple could be ignored in the final dataset.

4. Write a Conclusion section summarizing the study purpose, key findings for each RQ, and the most important limitation.

This study was conducted in order to find reasonings on whether software engineering teams should be picking Aspect or Object oriented programing when starting a new project. It was found that AOP projects had larger file sizes and were more complex, requiring larger teams to run as efficiently as OOP projects. This however means that OOP projects were not as useful for more complex tasks. It was also found that both had similar update rates, even if that may be influenced by team size and experience more so than the project type. The largest limitation of this study was the sample size, limiting the accuracy of the data, though the data was all normally distributed and removing an outlier often showed patterns for the two data types.